

★ WEEK 6 · LECTURE

Support Vector Machines

The widest road, the kernel trick — and OCSVM for anomalies

Expanded edition · 38 slides · with three hands-on labs

SVM finds the safest possible boundary between groups, can bend that boundary with a clever trick, and even draw a fence around 'normal' to catch anomalies.

Where We Are Going Today

Week 5 gave us 3 classifiers + LOF. SVM is a 4th classifier with a beautiful idea

1

The widest road (the margin)

Of all the lines that split two groups, SVM picks the one with the biggest safety buffer.

2

Messy data & curvy boundaries

Soft margin forgives a few mistakes; the kernel trick bends a straight line into a curve.

3

A fence around 'normal' (OCSVM)

Anomaly detector #2 — wrap the margin idea into a closed boundary.

By the end you will bend a boundary with a kernel and fence off 'normal' with OCSVM.

1

PART 1 • THE MARGIN

The Widest Road

SVM's big idea: maximize the margin

Which Line Is Best?

Many lines split the two groups — but they are not equally good

Picture two clusters of dots with a gap between them. You could draw many lines to split them. A line hugging one group is risky: a new point could easily land on the wrong side.

A line too close to one side

- Leaves almost no room for error.
- A slightly different new point crosses it.
- Fragile — it will make mistakes on fresh data.

SVM's answer

- Pick the line smack in the MIDDLE of the gap.
- Push both groups as far from the line as possible.
- The biggest buffer = the safest choice.

From Moscow to Bell Labs

Vladimir Vapnik carried the SVM idea across the world

"The idea simmered for decades in the Soviet Union, then exploded in the 1990s at Bell Labs — and SVM ruled ML for years."

Why it took over

- Before deep learning, SVM was the top performer on many tasks.
- It came with strong mathematical guarantees.
- Still a go-to for small, clean datasets today.

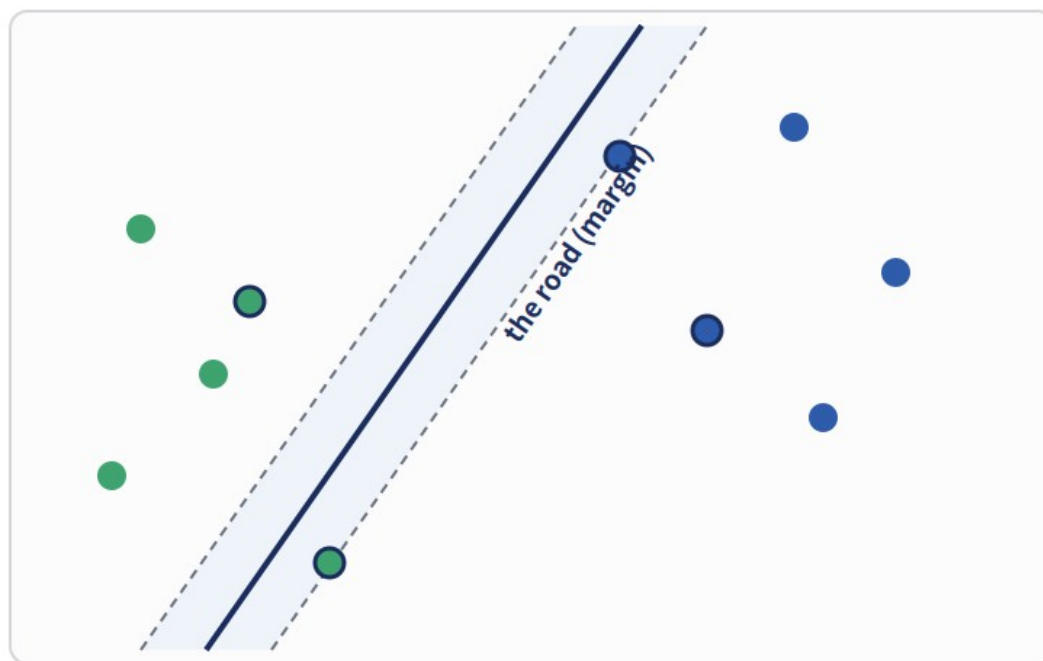
Why we learn it

- The 'widest margin' idea is deep and reusable.
- It powers our next anomaly detector (OCSVM).
- It teaches the kernel trick — a tool used everywhere.

The Widest Road & Support Vectors

Only the circled border points (support vectors) matter — the rest could vanish and nothing changes.

SVM — Build the Widest Possible Road Between Groups



The whole idea

Many lines can split the two colors.

SVM picks the one with the **WIDEST** empty road on both sides.

A wide buffer = more room for error
= better on new, unseen points.

The circled points touching the edges are the "support vectors".

Why 'Widest' Means 'Safest'

A wide buffer is forgiveness built into the model

Think of lane markings on a road. A wide lane lets you drift a little and stay safe. A razor-thin lane punishes the tiniest wobble. SVM builds the widest lane it can.

The payoff

- New points can sit a bit off and still be classified right.
- Fewer surprises on unseen data.
- This links straight back to bias-variance (Week 1).

Support vectors

- Only the points on the lane edges decide the boundary.
- Move a far-away point — nothing changes.
- So SVM is memory-light: it remembers just the border cases.

The Margin, as a Formula

Widening the road = shrinking a quantity called $\|w\|$

$$w \cdot x + b = 0 \quad (\text{the dividing line})$$

$$\min_{w, b} \frac{1}{2} \|w\|^2 \iff \text{widen the road}$$

Reading it

- Line 1 is the dividing boundary (line in 2D, plane in 3D).
- w controls the tilt; road half-width = $1 / \|w\|$.
- To WIDEN the road, shrink $\|w\|$.

In plain words

- Don't memorize the math — keep the picture.
- Smaller $\|w\|$ = wider safety road = safer model.
- Everything else in SVM is built on this one trade.

The Widest Road by Hand

Credit risk scores on a number line — five customers, one boundary (full version: problem set)

Customer	R1	R2	R3	D1	D2
Risk score	2	3	4	8	9
Outcome	repaid	repaid	repaid	default	default

The reasoning

- Any boundary b between 4 and 8 separates the groups.
- SVM refuses to hug either side: $b^* = (4+8)/2 = 6$.
- Margin = 2 score points of buffer on each side.

Support vectors

- Only R3 (score 4) and D1 (score 8) touch the road's edges.
- Move R1 to score 1 — the boundary does not care.
- Two points out of five carry the whole model.

2

PART 2 · SOFT MARGIN

When Data Is Messy

Soft margin: forgive a few mistakes

Hard vs. Soft Margin

Perfect separation is a fantasy — soft margin trades a few small errors for a wider, more robust boundary.

Hard vs. Soft Margin — Real Data Is Messy

Hard margin — strict fence

"No point may cross the line. Ever."

Works only if the two groups are perfectly separable.

One weird outlier forces a crazy, twisted boundary.

Real markets are never this clean, so hard margin overfits.

Soft margin — flexible fence

"Allow a few troublemakers across, to keep the road wide and simple."

Tolerates a few mistakes on purpose.

A knob called C sets how strict:
big C = strict, small C = forgiving.

This is what we always use in practice.
Trades a little error for a lot of stability.

Soft Margin & the C Knob

Add a penalty for each point that sneaks across

$$\min_{w, b} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i$$

Reading it

- First term: same 'widen the road' goal as before.
- Second term: add up the rule-breakers (the slack), times C.
- The model balances a wide road against a few crossings.

What C controls

- Big C: crossings are expensive → strict, narrow road (overfit risk).
- Small C: crossings are cheap → forgiving, wide road.
- Tuned on the validation set — it's the SVM cousin of Week 4's λ .

Three Settings of the C Knob

The forgiveness dial — Week 1's U-curve wearing yet another costume

C small — forgiving

wide road, some slack

Crossings are cheap, so the road stays wide and simple. A few oddballs are ignored.

noisy financial data's friend

C medium — balanced

the usual start

A reasonable trade between road width and rule-breakers. Tune from here on validation.

sklearn default $C = 1$

C huge — zero tolerance

narrow, brittle road

Every crossing is a crime, so the road contorts around single points. Student A returns.

overfitting in a new costume

Note the inversion: big $C \approx$ small λ (little regularization); small $C \approx$ big λ . Same U-curve, flipped axis.

One Oddball, Two Philosophies

A repaid customer shows up at score 7 — watch hard and soft margin react (full version: problem set)

Philosophy	Boundary b	Margin	Slack paid	Character
Hard margin (obey everyone)	$(7+8)/2 = 7.5$	0.5	0	brittle — one oddball crushed the road
Soft margin (forgive one)	stays at 6	2.0	$\xi = 7 - 4 = 3$	robust — paid 3 slack, kept 4× the road

Which wins? C decides: huge C makes slack unaffordable (hard); small C happily pays it (soft).

3

PART 3 · THE KERNEL TRICK

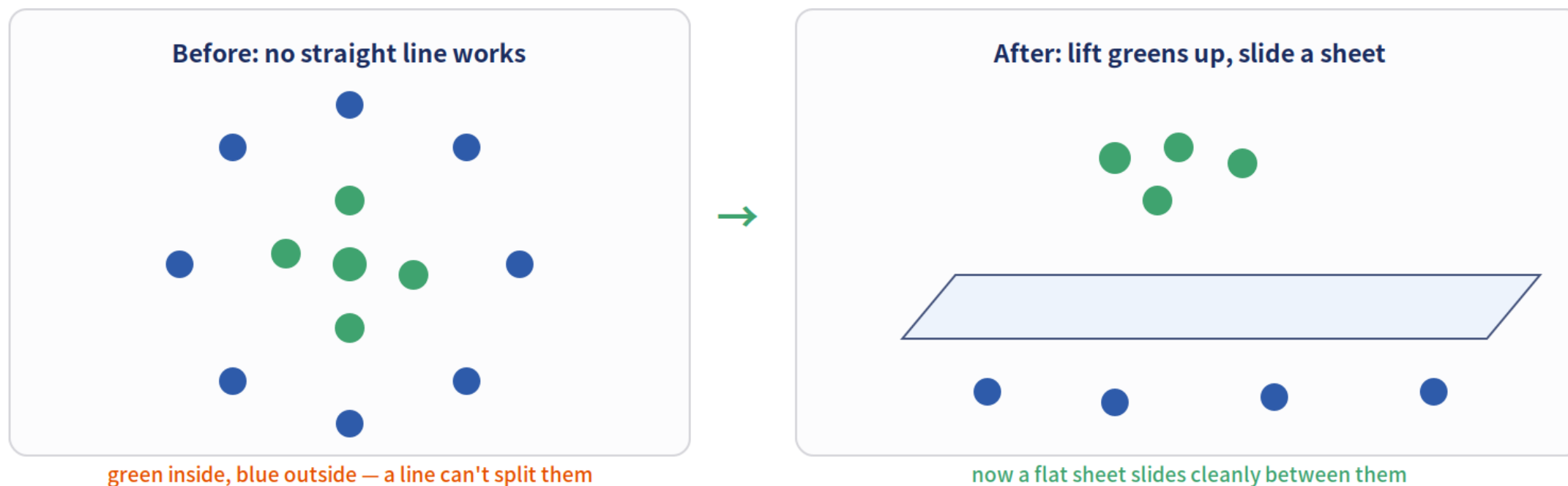
When a Line Won't Do

The kernel trick: bend the boundary

The Kernel Trick

Lift the data into a higher dimension where a flat boundary works — then project back to a curve.

The Kernel Trick — Lift the Data Until a Flat Sheet Fits



The kernel does this "lifting" with cheap math — it never actually builds the extra dimensions.

The RBF Kernel — the Popular Choice

Measures how 'close' two points are, smoothly

$$K(x, x') = \exp(-\gamma \|x - x'\|^2)$$

Reading it

- K measures similarity between two points x and x' .
- Very close $\rightarrow$ K near 1; far apart $\rightarrow$ K near 0.
- gamma sets how quickly similarity fades with distance.

Why it's magical

- Gives the 'lifting' effect without ever building extra dimensions.
- Curvy boundaries become cheap to compute — RBF is the default kernel.
- Big gamma = wiggly boundary; small gamma = smooth.

The Lift, by Hand — Three Points No Line Can Split

On a 1-D line, class B sits BETWEEN two class-A points. Lift with $z = x^2$ and watch

Point	x (original line)	Class	$z = x^2$ (lifted)	Now separable?
P1	-1	A	1	z above 0.5 → A side
P2	0	B	0	z below 0.5 → B side
P3	+1	A	1	z above 0.5 → A side

No boundary on the x-line separates A|B|A — but on the z-axis, the flat cut $z = 0.5$ does it perfectly. That is the lift; the kernel gets the same effect without ever computing z.

The γ Dial: How Fast 'Similar' Fades

Every kernel boundary is a similarity vote — γ sets the voters' eyesight

γ small — long sight

smooth boundary

Even far points look 'similar', so the boundary stays broad and calm. Underfits if too small.

safe start on noisy data

$\gamma = \text{'scale'}$ — auto

the sklearn default

Sets γ from the data's own spread. A sensible middle for most problems.

use unless you have a reason

γ huge — tunnel vision

wiggly boundary

Only near-twins count as similar; the boundary wraps single points. Overfitting again.

Student A, kernel edition

C and γ are tuned TOGETHER on validation — Lab 3 draws the full $C \times \gamma$ heat map.

SVR — the Same Idea for Numbers

Support Vector Regression predicts a value, not a class

Flip SVM around: instead of a road BETWEEN groups, draw a tube AROUND the trend line. Points inside the tube are 'close enough' and ignored; only the ones outside shape the fit.

How it differs from Week 4

- Linear regression cares about every point's error.
- SVR ignores small errors inside the tube.
- This makes it robust to noise — handy in finance.

A finance use

- Predicting volatility (how much a price will swing).
- Volatility is noisy; SVR's tube absorbs the wiggle.
- Same kernel trick applies for curvy patterns.

4

PART 4 · OCSVM

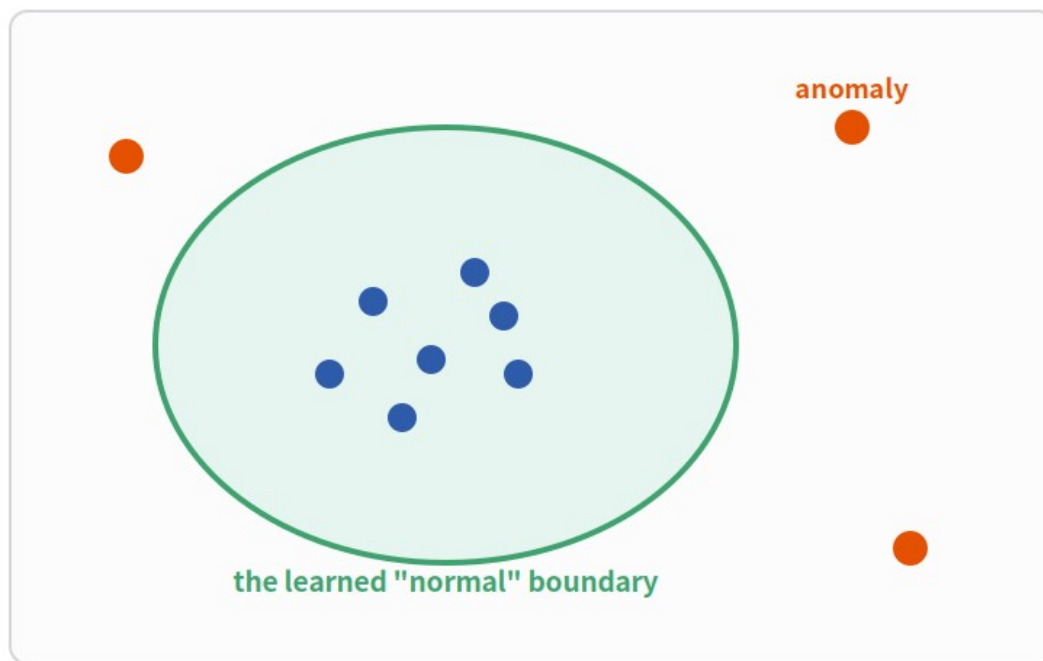
The Normal Boundary

OCSVM — anomaly detector #2

One-Class SVM — a Fence Around Normal

Show it only normal data; it learns the boundary that wraps them. Anything outside is flagged — no fraud labels needed.

One-Class SVM — Draw a Fence Around "Normal"



Anomaly detector #2

Show it only NORMAL examples.

It learns one boundary that wraps snugly around them.

Anything landing OUTSIDE the fence is flagged as an anomaly.

Same SVM margin idea — but now wrapped into a closed boundary.

OCSVM vs. LOF — Two Detectors, Two Styles

Both find anomalies, but they look at different things

LOF (Week 5)

- Idea: local density — is this point a loner?
- Compares each point to its own neighborhood.
- Great when normal data has dense + sparse regions; can be slow, sensitive to the neighbor count.

OCSVM (Week 6)

- Idea: one global fence around all normal points.
- Uses the SVM margin + kernel machinery.
- Handles high dimensions more stably; needs its own knob (ν) for the fence tightness.

In practice banks run several detectors together — each catches what the others miss.

The v Knob: How Tight Is the Fence?

v is roughly the share of training points allowed to sit outside

v too small (0.001)

- The fence stretches to include nearly every normal point.
- Even genuinely odd training points end up inside.
- New anomalies slip through — the fence is too generous.

v too big (0.2)

- The fence tightens; 20% of NORMAL points are pushed outside.
- The alarm rings constantly on honest customers.
- Rule of thumb: set v near your believed anomaly rate (~ 0.01 – 0.02).

v plays the same role as LOF's contamination — a stated belief about how rare 'odd' is.

The Anomaly-Detector Scoreboard, Updated

Two down, two to come — each sees 'odd' with different eyes

Detector	How it sees 'odd'	Strength	Its knob
LOF (W5)	emptier than your own neighborhood	mixed-density data	n_neighbors / contamination
OCSVM (W6)	outside the learned fence around normal	high dimensions, kernel-curved normal	ν (fence tightness), γ
iForest (W7)	easy to isolate with random cuts	speed on big data	coming next week
Autoencoder (W8)	poorly reconstructed by the network	complex, structured normal	coming in two weeks

Keep this table — by Week 8 you will own a four-detector fraud toolkit.

Where SVM Shows Up in Our Three Apps

One tool, many jobs across the course

Fraud detection

OCSVM

Draws a fence around normal spending; new, unusual transactions land outside.

joins LOF as a second backstop

AI chatbot

SVC on text

Turn a sentence into numbers (TF-IDF), then SVM sorts the intent: 'Balance?' vs 'Transfer?' vs 'Complaint?'

early chatbots ran on this

Markets

SVR

Predicts volatility from recent features; the tube absorbs noisy days, the kernel catches non-linear regimes.

robust curve-fitting

The margin idea is everywhere: classify, regress, or fence off the normal — same core trick.

Which Piece of SVM? — A 60-Second Drill

Cover the right column, answer out loud, then check yourself

Scenario	Which piece?
Two clean clusters, want the safest straight split	Plain SVM — maximize the margin
One weird-but-honest point crushes the road	Soft margin — pay a little slack (knob C)
Class B sits INSIDE a ring of class A	Kernel trick — lift, cut flat, project a curve
No fraud labels, only normal history	OCSVM — fence the normal (knob ν)
Boundary wraps every single training point	γ too big — tunnel vision, Student A again

If you got 4 of 5, the machinery of Week 6 is yours.

One U-Curve, Many Costumes

Every week hands you a dial — they are all the same bias-variance trade from Week 1

Week	The dial	Too loose ↔ too strict
W2	CUSUM threshold h	every wiggle rings ↔ alarm never rings
W3	barrier width (pt, sl)	noise labels ↔ everything labeled 0
W4	budget λ	factor-zoo overfit ↔ all betas starved to 0
W5	k neighbors · threshold	noise-chasing ↔ blurred groups
W6	$C \cdot \gamma \cdot v$	brittle wiggly fence ↔ everything 'normal'

Every dial is tuned the same way: sweep it, watch VALIDATION error, take the bottom of the U.

5

PART 5 · HANDS ON

Your Turn — Hands On

Bend a boundary with a kernel and fence off normal with OCSVM

Getting This Week's Data

Companion notebook — reuse the § 5 classification set

From the companion notebook (§ 5)

- Run Setup (§ 0), then the § 5 offline cell.
- Gives X (clues) and yb (label, ~1% positive).
- Same data as Week 5 — easy to compare, no login.

What we'll do

- Lab 1: classify with a linear vs an RBF SVM.
- Lab 2: wrap a fence around normal with OCSVM.
- All in scikit-learn, a few lines each.

[CODE] FML_Data_Setup_Guide.ipynb · § 5 offline classification set

```
from sklearn.datasets import make_classification
X, yb = make_classification(n_samples=20000, n_features=20, n_informative=6,
                           weights=[0.99, 0.01], random_state=42)
# X = clues, yb = label (about 1% positive, fraud-like)
```

Hands-On Lab 1 — Linear vs. RBF Kernel

See how bending the boundary changes the score

[CODE] colab_lab1.py · data from setup guide § 5

```
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import f1_score

# SVM is slow on big data -> use a sample
Xs, ys = X[:8000], yb[:8000]
Xtr, Xte, ytr, yte = train_test_split(Xs, ys, random_state=0)

for kern in ['linear', 'rbf']:
    m = SVC(kernel=kern, C=1.0, gamma='scale',
            class_weight='balanced') # 1% positives
    m.fit(Xtr, ytr)
    f1 = f1_score(yte, m.predict(Xte))
    print(f'{kern:6s} kernel -> F1 = {f1:.3f}')
```

Your task (step by step)

- Load X, yb (setup guide § 5).
- Take a small sample so SVM runs fast.
- Fit a linear SVM and an RBF SVM.
- Compare their F1 scores.
- Q: does the curvy RBF boundary help here?

Hands-On Lab 2 — OCSVM Fence Around Normal

Train on normal-only, then catch the anomalies

[CODE] colab_lab2.py · data from setup guide § 5

```
import numpy as np
from sklearn.svm import OneClassSVM

normal = X[yb == 0][:4000]      # train on normal only
oc = OneClassSVM(kernel='rbf', gamma='scale', nu=0.02)
oc.fit(normal)

flag = oc.predict(X[:8000])      # -1 = anomaly, 1 = normal
is_anom = (flag == -1)
caught = np.sum(is_anom & (yb[:8000] == 1))
print(f'OCSVM flagged {is_anom.sum()} points')
print(f'caught {caught} real anomalies',
      '(trained on normal only)')
```

Your task (step by step)

- Take only the normal rows ($y_b == 0$).
- Train OCSVM on those alone.
- Score ALL points; flag those outside the fence.
- Check how many real anomalies you caught.
- Q: how does this compare to LOF from Week 5?

Hands-On Lab 3 — Claude as Your Coding Copilot

AI is allowed in this course — it explains and extends; you understand and verify

[INPUT] Prompt · Claude (paste after your Lab 1 + 2 code)

You are my machine-learning tutor.

- 1) Explain my Lab 1 and Lab 2 code above, one plain-English sentence per line.
- 2) Grid-search the RBF SVC over C in $[0.1, 1, 10, 100]$ x γ in $[0.001, 0.01, 0.1, 1]$ on a validation split; draw the F1 heat map over the grid.
- 3) Mark the best cell and the 'Student A' corner (huge C , huge γ) and compare their TEST F1.
- 4) Sweep OCSVM ν in $[0.005, 0.01, 0.02, 0.05, 0.1]$: plot flags vs real catches, and pick a sensible ν .
- 5) Run Week 5's LOF on the same 8000 rows and report, in 3 sentences, where OCSVM and LOF disagree and why a bank would keep both.

House rules for AI use

- Allowed: explain, debug, extend code you wrote.
- Required: run it, read it, defend every line.
- Forbidden: submitting code you cannot explain.
- Tip: ask 'why', not just 'do'.

Comprehension Check

Try each question first — the answer is revealed below

Q1 What does an SVM try to maximize?

- A** The number of features
- B** The width of the road (margin) between groups ✓
- C** The training speed
- D** The number of support vectors

Answer: SVM picks the boundary with the widest empty buffer on both sides — the biggest margin.

Q2 What is the kernel trick for?

- A** Deleting weak features
- B** Separating groups no straight line can split ✓
- C** Speeding up Naive Bayes
- D** Labeling the data

Answer: The kernel lifts data into a higher dimension so a flat boundary works — curvy boundaries, cheaply.

Comprehension Check — Round Two

Two more — these cover the new slides

Q3 You crank **C** up to a huge value. The boundary becomes:

- A** Wider and more forgiving
- B** Narrow and contorted around single points ✓
- C** Exactly the same
- D** A perfect circle

Answer: Huge **C** makes every crossing a crime — the road narrows and twists to obey each point. Overfitting.

Q4 One-Class SVM is trained on:

- A** Fraud examples only
- B** Normal examples only ✓
- C** A 50/50 mix
- D** Unlabeled kernel weights

Answer: OCSVM learns the fence from normal data alone — anything falling outside is flagged, no fraud labels needed.

Week 6 on One Page

Photograph this slide — it is the whole lecture

Concept	One line to remember
Margin	the widest road between groups; support vectors are its edges.
Soft margin (C)	forgive a few crossings for a wider road — big C = strict = brittle.
Kernel trick	lift, cut flat, project back a curve — without building the lift.
RBF & γ	similarity that fades with distance; big γ = tunnel vision = wiggle.
OCSVM (ν)	fence around normal-only data; $\nu \approx$ how rare you believe 'odd' is.
SVR	the same margin as a tube around a trend — noise-robust regression.

Anomaly detector #3 (Isolation Forest) arrives next week — same fraud, different eyes.

Four Ideas, One Picture Each

Bonus — OCSVM: wrap the margin into a fence around 'normal' to flag anomalies (detector #2)

1

Margin = the widest road

Of all splitting lines, SVM picks the one with the biggest safety buffer. Only border points matter.

2

Soft margin = a forgiving fence

Allow a few crossings (knob C) for a wider, simpler boundary. Real data is never perfectly clean.

3

Kernel trick = bend the line

Lift the data so a flat sheet separates it — RBF gives curvy boundaries cheaply (knob γ).

Keep the pictures: the widest road, a forgiving fence, a lifted cut — and a fence around normal.

Week 6 — Three Things to Remember

Before next class: finish Labs 1–3 and compare OCSVM's catches with LOF's from Week 5

1

SVM maximizes the margin

The widest road between groups is the safest boundary. Support vectors (border points) define it.

2

Soft margin + kernel trick

C forgives a few mistakes for stability; the RBF kernel bends a straight line into a curve, cheaply.

3

OCSVM is anomaly detector #2

Learn a fence around normal; flag anything outside. SVR applies the same idea to predict numbers.

Next week — Trees & Ensembles: Decision Trees, Random Forest, XGBoost (+ Isolation Forest, #3).